

TOM

Topological Open Modular

Seeded Referential Substrate

Canonical Genome String and Formal Definition of a Proposed New Computing Paradigm

Tom Klootwijk

10-07-1990 | NL200678942 | 01/09/2026

Version: TOM-SRS 1.0

Core proposition

A finite canonical string is used as a content-addressed root that expands, under a fixed grammar and operator schedule, into typed definitions, state, bounded recursion, relation-zero events, transitions, and lineage. The authoritative result is a deterministic query trace; images, meshes, learned outputs, or physical embodiments are optional downstream phenotypes.

Requester-supplied attribution and identifiers; not independently verified. This document formalizes the supplied corpus. It is not

a patent grant, proof of priority, independent prior-art search, or validation of a physical device.

Document status and evidence boundary

This document condenses the supplied prompt chain and the later UGTS/SCLP formalizations into one proposed paradigm under the name **TOM** (Topological Open Modular). The earliest source gives the dense token sequence; later prompts ask for kinematic calculus inside the LUT, explicit operator precedence, generalized $SDF=0$ semantics, and a forward 1D time direction. A second source explicitly calls the token sequence a “latent vector seed” and a “self-contained, recursive programmatic genome.” [S1 p.1; S1 pp.8–20; S2 pp.3, 5, 10–12]

The later UGTS documents sharpen the proposal: definitions become typed, content-addressed substrate records; operation order becomes explicit; one-bit values remain selectors or bounded perturbations; topology is a declared gluing/routing rule; and the canonical output sequence is support → compatibility → guard → verified event → transition → lineage. [S3 pp.1–6, 9, 14; S4 pp.1–12]

Status of the novelty claim

The phrase “new paradigm” is used here as the formal name of the proposed architectural inversion. The supplied sources support a novel synthesis and an internally testable specification. They do not, by themselves, establish worldwide novelty, patentability, universal performance, or a new physical law. No independent prior-art search is included.

Contents

Document status and evidence boundary	i
1 Canonical genome string	1
1.1 Expansion diagram	1
2 How the initial prompts collapse into the seed	1
2.1 Overloaded symbols resolved	2
3 Why this is a seeded computational genome	2
3.1 Formal seed package	2
3.2 Genome analogy, stated precisely	3
4 Formal TOM substrate	3
4.1 World object	3
4.2 Definition node	3
4.3 Typed state	4
4.4 Determinism contract	4
5 Operator contract and precedence	4
5.1 Seed tokens	4
5.2 Normative phase order	5
6 Core mathematics encoded by the seed	5
6.1 Log-polar chart and kinematic calculus	5
6.2 One-bit jitter and parity	5
6.3 Finite cone, circle, sphere, and sweep	6
6.4 Time, phase, and winding	6
6.5 Klein gluing and chirality	6

6.6 Bounded binary grammar	6
7 The key operator: SDF0@Def	6
8 Query-first event semantics	7
8.1 Authoritative event condition	7
8.2 Minimum native queries	7
9 What makes the paradigm different	8
9.1 Relation to deterministic AI substitution	8
10 Minimal compiler and runtime contract	8
10.1 Canonicalization requirements	9
11 Falsifiability, acceptance, and limits	9
11.1 Acceptance tests	9
11.2 Claims deliberately not made	9
12 Final formal definition	10
A Compact seed grammar	11
B Source register	11

1 Canonical genome string

Authoritative one-line seed - UTF-8, ASCII token set

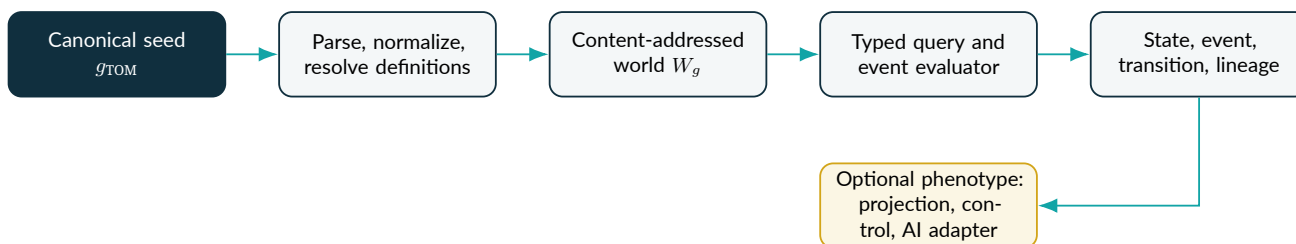
```
TOM1[TopologicalOpenModular]|TomKlootwijk|1990-07-10|NL200678942|2026-09-01|LUTlogp^{Klein,SDF0@Def}{rho,theta,t->;phi,dt,d2,J,v,a,j1}>P1>L2_BST^b>ASweepCone(T,apex)>Pi[pyrSide,circle,sphere]>support>compatibility>guard>event>transition>lineage
```

Canonical file	TOM_seed_genome_2026-09-01.txt
Encoding	UTF-8; ASCII characters only; no terminal line feed in the canonical byte sequence
Length	244 bytes
SHA-256	d1417a3136772c0cf3eddc4962ce07d42cbf87616f7b5bae09fc652d9b807b5
Pipeline reading	Left to right after the final metadata separator

One-sentence formal thesis

A **TOM** substrate is a deterministic, content-addressed interpreter of a finite seed into a typed state-and-relation world in which definitions, operator order, zero-relations, events, transitions, and lineage are first-class data, while rendering and learned behavior are optional outputs rather than the substrate itself.

1.1 Expansion diagram



The seed is not claimed to execute by linguistic magic. It becomes a computational genome only when paired with a versioned token dictionary, canonicalization rule, precedence schedule, resource budget, and deterministic evaluator. That package is formalized below.

2 How the initial prompts collapse into the seed

The source chain develops in a clear sequence. The compression keeps every recurring operator while discarding conversational scaffolding.

Stage	Source instruction	Canonical contribution
P0	“1bit parity bit lower case phi jitter log encoded polar LUT analytic sweeping cone T ... binary search tree L-system ... SDF klein bottle (sdf 0 on all definable points or bits)” [S1 p.1]	LUTlogp, phi, j1, P1, ASweepCone(T,apex), Pi[...], L2_BST^b, Klein, SDF0@Def.
P1	Add kinematic calculus to the earlier prompt. [S1 pp.2-4]	dt,d2,J,v,a; time-dependent state rather than static geometry.
P2	Formalize the prompts as one self-referential system. [S1 pp.4-6]	One root expression, closed operator registry, feedback and trace semantics.

Stage	Source instruction	Canonical contribution
P3	Chain the prompt as operators, pack kinematics into the LUT, and make precedence explicit. [S1 pp.8–10]	A deterministic pipeline rather than a bag of metaphors.
P4	Move SDF=0 from one apex boundary into the LUT as a general definitional operator. [S1 pp.15–18]	SDF0@Def: every definition exposes a zero-relation interface.
P5	Add delta-delta, lowercase phi, and forward 1D timeline adherence under TOM. [S1 pp.18–22; S2 pp.5–7]	t->, phi, d2, and the umbrella name TOM .
Mature formalization	Definitions live in the substrate; references resolve; hashes verify; operation order is explicit; the result flows through support, compatibility, guard, event, transition, and lineage. [S3 pp.1–6, 9; S4 pp.9–12]	Final query-first tail of the canonical string.

2.1 Overloaded symbols resolved

The early dialogue sometimes assigned several meanings to the same symbol. The canonical seed follows the later SCLP contract so that the genome has one deterministic parse. [S4 p.2]

Token	Canonical meaning	Excluded ambiguity
T	Finite cone slant length	Not global time, torque, or bit shift
t / t->	Continuous time with monotone causal evaluation order	Not a claim that time ceases to be linear
phi	Lowercase $\phi \in S^1$, a phase/hinge coordinate	Not automatically the golden ratio Φ
BST	Conceptual binary branch organization	Fixed-width key refinement is a radix-2 trie, not a comparison tree
SDF0@Def	Per-definition zero relation or event surface	Not a constant-zero field and not one bit containing a full SDF
Klein	Explicit orientation-reversing gluing and sheet/orientation transport	Not a claim that every projected self-intersection physically couples

3 Why this is a seeded computational genome

3.1 Formal seed package

Let the canonical string be $g \in \Sigma^*$. The actual genome is the finite package

$$\mathcal{G}_{\text{TOM}} = (g, \nu, h_g, \Gamma, \mathcal{P}, \mathcal{B}),$$

where ν is the version, h_g the content hash, Γ the typed token dictionary, $\mathcal{P}$ the phase-ordered evaluator, and $\mathcal{B}$ the explicit depth, symbol, memory, numeric-error, and event budgets. Compilation is

$$\text{Compile}(\mathcal{G}_{\text{TOM}}) = (W_g, \text{Eval}_g).$$

This separates the compact genotype from the expanded executable phenotype.

3.2 Genome analogy, stated precisely

Genome concept	TOM object	Technical meaning
Sequence	Canonical string g	Stable, hashable root; operator order is present in the sequence.
Gene	Typed definition node	A named operator, relation, topology map, profile, or schedule with domain and codomain.
Regulatory order	Evaluation phase and dependencies	Controls when an operator may be expressed; composition is not silently commuted.
Development	Bounded L-system / grammar	Expands a compact rule set into finite branch structure under declared budgets.
Mutation / variation	Deterministic one-bit jitter and branch bit	Reproducible local variation or routing, bounded by an event margin; not arbitrary hidden state.
Inheritance	Content hash and lineage	Preserves definition identity and traces ancestry across transitions.
Environment	Query, input, and context	Selects supports, compatible sectors, event candidates, and transition parameters.
Viability gate	Support $\rightarrow$ compatibility $\rightarrow$ guard	Rejects expressions or events that do not satisfy the declared local conditions.
Phenotype	State/event trace; optional projection	What the seed produces for a particular context, without changing the genome.

Self-reference without uncontrolled circularity

The substrate may carry definitions of its own operators, schedules, and verification rules. That is referential closure. Evaluation dependencies remain acyclic and hash-verifiable, while recursion is confined to a separately budgeted grammar. Self-description is therefore allowed; unbounded circular execution is not required. This matches the later UGTS rule that definitions are first-class substrate nodes and the dependency graph must admit a deterministic order. [S3 pp.3–4, 11–14]

4 Formal TOM substrate

4.1 World object

The compiled world is

$$W_g = (D, X, G, R, S, C, H, E, \mathcal{T}, I, L, \mathcal{P}, g, h_g),$$

with the following typed components:

- D : in-substrate definition registry;
- X : literal instances and state seeds;
- G : finite, budgeted grammar;
- R : scalar, interval, or symbolic relations;
- S : local support predicates;
- C : compatibility predicates;
- H : continuous, discrete, and connector hinges;
- E : verified event records;
- $\mathcal{T}$: atomic transition maps;
- I : invariants and fixed subspaces;
- L : lineage and exogenous novelty log;
- $\mathcal{P}$: phase-ordered pipelines.

This extends the UGTS 3.6 world object by attaching the canonical root and its hash. [S3 p.3]

4.2 Definition node

A definition is a first-class record

$$d = (id, kind, A, B, \vartheta, Dep, phase, cap, inv, prov, h),$$

where A and B are domain and codomain, ϑ is the literal parameter set, Dep the dependency IDs, cap the declared capabilities, inv invariants, $prov$ provenance class, and

$$h = \text{SHA256}(\text{canonicalJSON}(d \setminus \{h\})).$$

Instances reference definitions; pipelines reference definition IDs; unresolved IDs, bad hashes, or undeclared cycles invalidate compilation. [S3 pp.3–4, 11–12]

4.3 Typed state

A useful SCLP-complete state is

$$q = (x, t, \rho, \theta, \phi, \dot{\phi}, \ddot{\phi}, o, s, b, \omega_G, K, A_g, \lambda, u),$$

where x is position, (ρ, θ) a local log-polar chart, ϕ the hinge/phase angle, $o \in \{-1, +1\}$ orientation, s a sheet, b a branch or selector bit, ω_G a finite grammar word, K a finite address, A_g a generative address, λ lineage, and u uncertainty. [S4 p.2]

4.4 Determinism contract

For a fixed genome version, definition registry, input, query, seed, tolerance, and budget,

$$\text{Eval}_g(Q, \text{input}) = \text{trace}$$

must return the same typed trace and terminal certificate on every conforming CPU or GPU implementation. Parallel scheduling may differ; observable operator order, reductions, rounding profile, and emitted events may not.

5 Operator contract and precedence

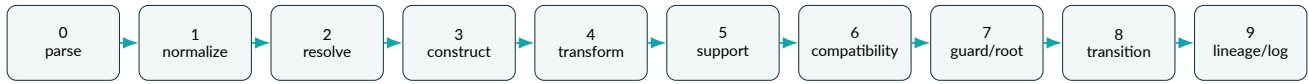
5.1 Seed tokens

Seed token	Type	Normalized operation
<code>LUTlogp(rho, theta, ...)</code>	Coordinate/state index	Log-polar chart and finite lookup/key domain with an explicit singular core.
<code>Klein</code>	Topology profile	Orientation-reversing wrap, sheet/orientation transport, and lineage update.
<code>SDF0@Def</code>	Relation interface	Attach a zero-locus/event relation to every eligible definition.
<code>phi</code>	Periodic scalar	Hinge/phase coordinate in S^1 .
<code>dt, d2, J, v, a</code>	Kinematic family	Time increment, change-of-change, Jacobian, velocity, and acceleration.
<code>j1</code>	Bounded bit perturbation	Deterministic signed dither or selector with an interval certificate.
<code>P1</code>	One-bit parity/hinge	Modulo-two reduction selecting one of two declared routes; not full state.
<code>L2_BST^b</code>	Grammar/router	Bounded bifurcating L-system organized by a binary branch bit; address refinement uses a radix trie.
<code>ASweepCone(T, apex)</code>	Geometric relation family	Finite analytic cone plus a certified sweep relation.
<code>Pi[pyrSide, circle, sphere]</code>	Projection/support family	Triangular meridian profile, circular base, and spherical local support; optional downstream view.
<code>support</code>	Predicate	Admit only locally relevant relation candidates.

Seed token	Type	Normalized operation
compatibility	Predicate	Require matching phase, sheet, orientation, address, or declared coupling law.
guard	Certified relation test	Detect a valid zero crossing or guarded relation state.
event	Immutable record	Capture time, relation, direction, margin, confidence, route, and pre-state.
transition	Atomic map	Apply the declared post-event state patch.
lineage	Persistence	Append ancestry, winding, definition hashes, and irreducible novelty.

5.2 Normative phase order

The compact seed expresses the domain operators. A conforming evaluator expands them into the following ten phases, preserving the later UGTS operation schedule. [S3 pp.9–10]



Operator composition is generally non-commutative. A topology transition is not the same operation as a continuous field morph; a branch decision is not taken before its guard; a transition is not committed before support and compatibility are satisfied.

6 Core mathematics encoded by the seed

6.1 Log-polar chart and kinematic calculus

For a declared core scale $r_0 > 0$,

$$\rho = \ln(r/r_0), \quad \theta = \text{atan2}(y, x), \quad r = r_0 e^\rho.$$

The origin remains singular and must be represented by an explicit core flag. The planar metric is

$$ds^2 = r^2(d\rho^2 + d\theta^2), \quad g(\rho) = r^2 I_2,$$

and a finite log-radius step is

$$\Delta r = r(e^{\Delta\rho} - 1).$$

With radial and angular unit vectors e_r, e_θ ,

$$v = r(\dot{\rho}e_r + \dot{\theta}e_\theta),$$

$$a = r \left[(\ddot{\rho} + \dot{\rho}^2 - \dot{\theta}^2)e_r + (\ddot{\theta} + 2\dot{\rho}\dot{\theta})e_\theta \right].$$

These equations are the precise content of $\text{dt}, \text{d2}, \text{J}, \text{v}, \text{a}$ in the log-polar LUT. [S4 pp.3–4]

6.2 One-bit jitter and parity

The jitter bit and the parity bit are separate typed roles. For genome seed, finite address K , and tick X ,

$$b_j = H(g, K, X) \bmod 2, \quad \sigma = 2b_j - 1,$$

$$R_j(q) = R(q) + \epsilon_j \sigma, \quad R(q) \in [R_j(q) - \epsilon_j, R_j(q) + \epsilon_j].$$

The perturbation is admissible only when the interval cannot alter the certified event class or ordering, typically $\epsilon_j < \epsilon_g$. A parity operator may be

$$p = \text{popcount}(K \parallel \hat{q}) \bmod 2,$$

and selects one of two declared maps. It detects or routes a parity condition; it is not a complete error-correcting code and does not contain the continuous state. [S4 p.5; S3 pp.2, 13]

6.3 Finite cone, circle, sphere, and sweep

A cone is not determined by slant length alone. With slant length $T > 0$ and half-angle $\alpha \in (0, \pi/2)$,

$$h = T \cos \alpha, \quad R = T \sin \alpha.$$

The meridian is a filled triangle; rotation about its axis gives the circular solid. The seed's `pyrSide` and `circle` are therefore two projections of the same finite relation, while `sphere` supplies a separate local radial support. [S4 pp.2-3]

For a translational path $s(u)$, $u \in [0, 1]$,

$$F_{sweep}(x) = \inf_{u \in [0,1]} d_C(x - s(u)).$$

For a sampled linear sweep of length L and sample minimum m_n ,

$$m_n - \frac{L}{2(n-1)} \leq F_{sweep}(x) \leq m_n.$$

Thus the seed names a certified relation family, not an automatic exact SDF for every rotated or grammar-generated sweep. [S4 p.3]

6.4 Time, phase, and winding

The token $\mathfrak{t} \rightarrow$ means monotone causal evaluation and append-only event order. Linear time is retained. For reference tick X_0 and period P ,

$$u = \frac{X - X_0}{P}, \quad w = \lfloor u \rfloor, \quad \psi = u - w \in [0, 1).$$

Phase ψ may repeat; winding w and lineage keep repeated coordinates distinguishable. [S4 p.5]

6.5 Klein gluing and chirality

The source half-turn map and a genuine orientation-reversing base quotient must be kept distinct. A normalized odd-wrap Klein profile is

$$(\rho, \theta, \phi, o, s) \mapsto (\rho', \pi - \theta, -\phi, -o, s \oplus 1).$$

The reflection reverses angular orientation. This is coordinate and state transport, not an instantaneous physical torque impulse. [S4 pp.5-6]

6.6 Bounded binary grammar

A schematic branch production is

$$F(T) \Rightarrow F(T/2) [\pm \phi F(T/2)] J(\Delta r),$$

where the sign is selected by the branch bit and may be reversed by topological chirality. Depth, symbol count, stack depth, and expression size are finite declared budgets. [S4 p.7]

7 The key operator: SDF0@Def

The literal source phrase is “SDF 0 on all definable points or bits.” Taken as the constant equation $f(q) = 0$ for every q , it would erase all discrimination: no inside/outside sign, no residual, no crossing, and therefore no event information. The only non-degenerate formalization is to make zero a universal interface, not a universal value.

Universal zero-relation interface

For every eligible definition $d \in D$, the substrate exposes a typed relation

$$R_d : Q \times Context \rightarrow \mathbb{R} \text{ or } \mathbb{IR},$$

with zero locus

$$Z_d = \{q \in Q \mid 0 \in R_d(q)\}.$$

The notation $SDF@Def$ means “every definition may be queried through its declared zero relation.” It does not mean that every stored scalar, address, or bit is numerically zero.

This preserves the user’s intended inversion: the boundary/event relation is not an external afterthought applied only to one apex. It is a common capability of definitions inside the LUT/registry. Geometry, topology, grammar, compatibility, and even definition verification may each expose a relation whose zero state has typed semantics.

Examples:

- Geometry: $R_d(q) = 0$ is a surface or contact relation.
- Compatibility: $R_d(q) = 0$ is a mismatch residual reaching admissibility.
- Topology: $R_d(q) = 0$ is a wrap port or gluing boundary.
- Grammar: $R_d(q) = 0$ is a budget frontier or terminal production condition.
- Definition integrity: $R_d(q) = 0$ is zero hash discrepancy.

The word *SDF* is retained for exact signed-distance-capable geometric definitions. Generic definitions use the broader term *relation*. This distinction is required by the later UGTS/SCLP evidence discipline. [S3 pp.2, 4; S4 pp.10–12]

8 Query-first event semantics

8.1 Authoritative event condition

For trajectory/state expression $q(t)$, candidate relation R_k , local support S_α , compatibility χ , and confidence threshold,

$$t^* = \min \{t > t_0 : 0 \in R_k(q(t)), S_\alpha(q(t), t) = 1, \chi(q(t), k, t) = 1, \text{conf} \geq \text{conf}_{\min}\}.$$

The event record contains at least the definition hash, pre-state, event time, relation residual or interval, crossing direction, solver status, guard margin, confidence, route, and lineage. The transition is then committed atomically:

$$q(t^{*+}) = \mathcal{T}_k(q(t^{*-}), \text{context}),$$

followed by a lineage append. [S3 p.9; S5 pp.5–7]

8.2 Minimum native queries

A TOM implementation should answer these symbolic queries without requiring a rendered frame:

- `definition_at(id)` and `verify_definition(id)`;
- `state_at(t)` for a fixed closed expression;
- `next_event(t0)` under support and compatibility;
- `events_in_support(cone, interval)`;
- `compatible(q1, q2)` across phase, sheet, orientation, and address;
- `trace(pipeline, instance)` and `reconstruct(lineage)`.

The mature corpus identifies this as the central shift: the substrate asks what state holds, what event occurs next, and which sectors may couple. Rasterization, ray marching, or AI generation may consume those results, but they do not define the world. [S5 pp.1, 5–12]

9 What makes the paradigm different

Question	Conventional frame/object pipeline	TOM seeded substrate
Where are the laws?	In source code, model weights, engine conventions, and undocumented order.	In typed, content-addressed definition nodes inside the substrate.
What is primary?	Objects, buffers, frames, pixels, or sampled tensors.	Relations, supports, compatibility, events, transitions, and lineage.
How is identity kept?	Current coordinate, pointer, object ID, or snapshot.	Generative address plus invariant and lineage history.
How does time advance?	Repeated global stepping or replay of stored states.	Closed state evaluation and certified next-event solving where the relation family allows it.
What is topology?	Often implicit in mesh connectivity or object code.	Explicit gluing, orientation, sheet transport, and route maps.
What does one bit mean?	Often an overloaded packed state or visual sample.	A narrow parity, branch, predicate, or bounded perturbation role.
What is rendered?	The stored world is projected every frame.	Projection is optional phenotype; symbolic query output is authoritative.
What is remembered?	Frames, dense grids, or full histories.	Seed, definitions, invariant state, lineage, and irreducible exogenous novelty.

Paradigm assertion

The proposed novelty is the combination of: (1) a hash-stable textual root; (2) definitions as substrate data; (3) bounded grammatical expansion; (4) per-definition zero-relation semantics; (5) local support and compatibility before event solving; (6) atomic transitions; and (7) lineage-based persistence. The system is therefore a *definition-and-event substrate*, not merely a graphics formula or compressed prompt.

9.1 Relation to deterministic AI substitution

The substrate can replace a learned model only for tasks whose behavior can be expressed by its definitions, grammar, relations, and transition rules. In that domain it provides deterministic procedural inference: identical seed and context yield identical trace. It does not automatically learn unknown semantics, infer an undeclared world model, or reproduce the open-ended capability of a trained general model. A learned system may be an adapter, teacher, or optional phenotype; it is not required by the TOM core.

10 Minimal compiler and runtime contract

Listing 1: Reference semantics; implementation language is not prescribed.

```

compile(seed_bytes, registry, profile, budgets):
    assert sha256(seed_bytes) == declared_seed_hash
    tokens = parse_exact(seed_bytes)
    ast = typecheck(tokens, profile.token_dictionary)
    definitions = resolve_all(ast, registry)
    verify_content_hashes(definitions)
    order = topological_order(definitions.dependencies)
    return World(seed, definitions, order, profile, budgets)

query(world, initial_state, request, context):
    q = construct_typed_state(initial_state)
    q = apply_logpolar_kinematics_and_bounded_jitter(q)
    q = apply_parity_hinge_and_bounded_grammar(q)
    relations = build_cone_sphere_topology_relations(q)

```

```

candidates = filter_support(relations, request)
candidates = filter_compatibility(candidates, q, context)
event = solve_earliest_certified_zero(candidates, q, context)
q_next = atomic_transition(event, q, context)
lineage = append_lineage(world.seed_hash, event, q_next)
return certificate(q_next, event, lineage, optional_projection(request))

```

10.1 Canonicalization requirements

A conforming implementation records at least:

- exact seed bytes and SHA-256;
- profile/version IDs and definition hashes;
- numeric type, rounding rule, angle normalization, and interval policy;
- deterministic hash/PRNG algorithm for j1;
- grammar and event budgets;
- stable reduction order where floating-point non-associativity matters;
- full trace of support, compatibility, guard, transition, and lineage decisions.

The same semantics can run on a CPU, GPU, FPGA, or another deterministic compute target. Hardware changes scheduling and throughput, not the meaning of the seed.

11 Falsifiability, acceptance, and limits

11.1 Acceptance tests

The paradigm has been instantiated correctly for a target domain only when all of the following are demonstrated:

1. The exact seed and all definitions hash reproducibly.
2. Every token has one typed domain, codomain, phase, and dependency set.
3. Grammar expansion terminates within declared budgets.
4. Jitter intervals cannot change a certified event class or event ordering.
5. Support and compatibility remove enough candidates to justify query-first execution.
6. `state_at(t)` and `next_event` match a trusted baseline within the declared error model.
7. Topological wraps preserve declared invariants and update orientation/sheet/lineage correctly.
8. Address codecs round-trip exactly where finite packed keys are used.
9. Results are deterministic across repeated runs and conforming devices.
10. Any performance claim is measured against a conventional method at equal semantics and error.

11.2 Claims deliberately not made

Technical boundary

TOM-SRS 1.0 does not assert infinite precision in finite memory, universal $O(1)$ execution, zero-cost anti-aliasing, automatic error correction from one parity bit, a constant-zero field that still stores arbitrary information, physical energy or force generated by topology, a validated quantum/optical device, or proof that the architecture is the first such idea worldwide. Those are separate hypotheses requiring models, measurements, and prior-art research.

These boundaries do not weaken the core claim. They make it testable: a compact, content-addressed seed can define a deterministic query/event substrate, and that substrate can be compared against frame-stepped or materialized baselines on a controlled relation family.

12 Final formal definition

TOM-SRS 1.0

TOM Seeded Referential Substrate 1.0 is a finite, content-addressed, query-first computing specification rooted in the canonical 244-byte genome string of 01/09/2026. The seed resolves into typed operator definitions and a phase-ordered evaluator. Its local state is indexed through a finite log-polar profile with explicit kinematics, bounded one-bit jitter, and a parity/branch hinge. A bounded binary grammar organizes analytic cone, circular, spherical, projection, and topological relations. Every eligible definition exposes a typed zero-relation interface; support and compatibility gate the earliest certified event; transitions are atomic; identity persists through hashes, generative address, winding, invariants, and lineage. Projection, control, learned inference, and physical hardware are adapters over the substrate rather than its authority.

The reason the condensed string functions as a genome is therefore structural, not rhetorical. It contains the names of the coordinate system, variation mechanism, kinematics, branching law, geometric constructors, topology, zero-event interface, query gates, transition semantics, and inheritance mechanism in executable order. The seed is small; the versioned definition graph is its interpreter; the event trace is its expressed form.

A Compact seed grammar

The following EBNF is a minimal deterministic grammar for the delivered one-line seed. Literal punctuation is significant.

```
seed      = header, "|", pipeline ;
header    = "TOM1[TopologicalOpenModular]", "|",
           "TomKlootwijk", "|", "1990-07-10", "|",
           "NL200678942", "|", "2026-09-01" ;
pipeline  = lut, ">P1>L2_BST^b>", cone, ">", projection,
           ">support>compatibility>guard>event>transition>lineage" ;
lut       = "LUTlogp^{Klein,SDF0@Def}(rho,theta,t->";
           "phi,dt,d2,J,v,a,j1)" ;
cone      = "ASweepCone(T,apex)" ;
projection = "Pi[pyrSide, circle, sphere]" ;
```

Token meanings are resolved by the TOM-SRS 1.0 registry—not by unrestricted natural-language interpretation. Any semantic or precedence change requires a new version token and content hash.

B Source register

ID	Registered source	Role in this synthesis	SHA-256 prefix
S1	Initial TOM operator-chain dialogue (30-page export)	Original dense prompt; kinematic extension; explicit precedence; SDF0@Def; forward-time and hardware motifs.	ede0fd37504b07fa
S2	Expanded TOM seed dialogue (12-page export)	Latent-token framing; explicit “programmatic genome” rationale; operator-by-operator unpacking.	be77ac39e9b77d8d
S3	UGTS_KC_3_6	Literal definition registry; content addressing; world object; normative operation order; evidence discipline.	e21433cd8378368d
S4	UGTS_KC_3_6_2_SCLP	Typed symbol contract; cone and sphere relations; log-polar kinematics; jitter; topology; grammar; finite packing.	6987490a0a7b7ed7
S5	Chronological spherical-substrate synthesis	Query-first interpretation; state/event semantics; feasibility framing; projection kept downstream.	9bacfe071a64951f

Exact local filenames are preserved in the accompanying project archive; this register uses stable source IDs and hash prefixes so that long filenames do not become part of the formal operator syntax.

Provenance note

The source files include earlier AI-generated explanations that contain useful motifs and technical overstatements. This document preserves the literal prompt vocabulary, identifies the source-derived reasoning, and uses the later UGTS/SCLP specifications to supply typed, finite, and falsifiable semantics. The normalization is explicit rather than silent.

Canonical seed SHA-256

d1417a3136772c0cf3edddcd4962ce07d42cbf87616f7b5bae09fc652d9b807b5

TOM-SRS 1.0 - 01/09/2026